

FCFS:

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[10], wt[10], tat[10]; // burst, waiting, turnaround
    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        printf("Enter burst time of P%d: ",i+1);
        scanf("%d",&bt[i]);
    }

    wt[0] = 0; // first process waiting time is 0

    // Calculate waiting time
    for(i=1;i<n;i++){
        wt[i] = wt[i-1] + bt[i-1];
    }

    // Calculate turnaround time
    for(i=0;i<n;i++){
        tat[i] = wt[i] + bt[i];
    }

    printf("Process\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\n",i+1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

SJF:

```
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int bt[10], wt[10], tat[10], p[10]; // bt: burst time, wt: waiting time, tat: turnaround, p: process ID

    printf("Enter number of processes: ");
    scanf("%d",&n);
```

```

for(i=0;i<n;i++){
    p[i] = i+1; // assign process IDs P1, P2, ...
    printf("Enter burst time of P%d: ", p[i]);
    scanf("%d",&bt[i]); // input burst times
}

// Sort processes by burst time (smallest first) along with process IDs
for(i=0;i<n-1;i++){
    for(j=i+1;j<n;j++){
        if(bt[i] > bt[j]){
            // Swap burst times
            temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            // Swap process IDs to match
            temp = p[i]; p[i] = p[j]; p[j] = temp;
        }
    }
}

wt[0] = 0; // first process waiting time is 0

// Calculate waiting times
for(i=1;i<n;i++){
    wt[i] = wt[i-1] + bt[i-1];
}

// Calculate turnaround times
for(i=0;i<n;i++){
    tat[i] = wt[i] + bt[i];
}

// Print table
printf("\nProcess\tBT\tWT\tTAT\n");
for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
}

return 0;
}

```

Priority Scheduling:

(Non Preemptive)

```
#include <stdio.h>
```

```
int main() {
```

```

int n,i,j,temp;
int bt[10], pr[10], wt[10], tat[10];

printf("Enter number of processes: ");
scanf("%d",&n);

for(i=0;i<n;i++){
    printf("Burst time of P%d: ",i+1);
    scanf("%d",&bt[i]);
    printf("Priority of P%d: ",i+1);
    scanf("%d",&pr[i]);
}

// sort based on priority (lower number = higher priority)
for(i=0;i<n-1;i++){
    for(j=i+1;j<n;j++){
        if(pr[i] > pr[j]){
            // swap priority
            temp = pr[i];
            pr[i] = pr[j];
            pr[j] = temp;
            // swap burst time accordingly
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
        }
    }
}

wt[0] = 0;
for(i=1;i<n;i++){
    wt[i] = wt[i-1] + bt[i-1];
}

for(i=0;i<n;i++){
    tat[i] = wt[i] + bt[i];
}

printf("Process\tBT\tPR\tWT\tTAT\n");
for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\n",i+1, bt[i], pr[i], wt[i], tat[i]);
}

return 0;

```

```
}
```

Priority Scheduling: (Preemptive)

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main() {
    int n, i, j, time = 0, completed = 0;
    int bt[10], at[10], pr[10], rt[10], wt[10], tat[10];
    int min_pr, idx;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i=0;i<n;i++){
        printf("Enter arrival time of P%d: ", i+1);
        scanf("%d", &at[i]);
        printf("Enter burst time of P%d: ", i+1);
        scanf("%d", &bt[i]);
        printf("Enter priority of P%d (lower number = higher priority): ", i+1);
        scanf("%d", &pr[i]);
        rt[i] = bt[i]; // remaining time initially = burst time
    }

    // Initialize waiting and turnaround times
    for(i=0;i<n;i++){
        wt[i] = 0;
        tat[i] = 0;
    }

    printf("\nGantt Chart:\n");

    // Loop until all processes complete
    while(completed < n){
        min_pr = INT_MAX;
        idx = -1;

        // Find process with highest priority at current time
        for(i=0;i<n;i++){
            if(at[i] <= time && rt[i] > 0){
                if(pr[i] < min_pr){
```

```

        min_pr = pr[i];
        idx = i;
    }
    // if same priority, pick the one that arrived first
    else if(pr[i] == min_pr && at[i] < at[idx]){
        idx = i;
    }
}
}

if(idx != -1){
    printf("P%d | ", idx+1); // Gantt chart output
    rt[idx]--; // execute process for 1 unit time
    time++;

    // If process finishes
    if(rt[idx] == 0){
        completed++;
        tat[idx] = time - at[idx]; // turnaround = completion - arrival
        wt[idx] = tat[idx] - bt[idx]; // waiting = turnaround - burst
    }
} else {
    // CPU idle
    printf("IDLE | ");
    time++;
}
}

printf("\n\nProcess\tAT\tBT\tPR\tWT\tTAT\n");
for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], pr[i], wt[i], tat[i]);
}

return 0;
}

```

Round Robin:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, quantum, time = 0, remain;
```

```
    int bt[10], rt[10], wt[10], tat[10];
```

```
    printf("Enter number of processes: ");
```

```

scanf("%d",&n);

for(i=0;i<n;i++){
    printf("Burst time of P%d: ",i+1);
    scanf("%d",&bt[i]);
    rt[i] = bt[i]; // remaining time
}

printf("Enter time quantum: ");
scanf("%d",&quantum);

remain = n;
while(remain != 0){
    for(i=0;i<n;i++){
        if(rt[i] > 0){
            if(rt[i] > quantum){
                time += quantum;
                rt[i] -= quantum;
            } else {
                time += rt[i];
                wt[i] = time - bt[i];
                rt[i] = 0;
                remain--;
            }
        }
    }
}

for(i=0;i<n;i++){
    tat[i] = bt[i] + wt[i];
}

printf("Process\tBT\tWT\tTAT\n");
for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\n",i+1, bt[i], wt[i], tat[i]);
}

return 0;
}

```